

INS Project Report: ChatAppV2

1. Cover Page

=====

PROJECT REPORT ON INFORMATION & NETWORK SECURITY

=====

ChatAppV2

A Real-Time Messaging Platform with Custom Double-RSA

End-to-End Encryption (E2EE)

=====

Submitted in partial fulfillment

of the requirements for the course of

INFORMATION AND NETWORK SECURITY LABORATORY (21CS62)

Submitted By:

Student NameUSN

[Student Name 1][USN 1]

[Student Name 2][USN 2]

Under the Guidance of:

[Guide Name, Title]

Department of CSE, [College Name]

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

[INSTITUTE/COLLEGE NAME]

[ACADEMIC YEAR: 2025-2026]

=====

2. Certificate Page

=====

[INSTITUTE/COLLEGE NAME]

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

=====

CERTIFICATE

Certified that the project work titled "ChatAppV2: A Real-Time Messaging Platform

with Custom Double-RSA End-to-End Encryption" is a bonafide work carried out by [Student Name 1] ([USN 1]) and [Student Name 2] ([USN 2]) in partial fulfillment of the requirements for the Information and Network Security Laboratory (21CS62) course during the Academic Year 2025-2026.

The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the degree of Bachelor of Engineering in Computer Science & Engineering.

[Guide Name]
Project Guide
Department of CSE
[Institute Name]

[HOD Name]
Head of Department
Department of CSE
[Institute Name]

External Examiner 1
Signature & Date

External Examiner 2
Signature & Date

3. Abstract

Problem Statement

In contemporary digital communications, absolute privacy is highly compromised by centralized messaging systems. Standard client-server architectures routinely intercept, store, and analyze user conversation metadata and message payloads under the pretext of server-side validation or message history preservation. While some applications offer End-to-End Encryption (E2EE), they often store keys insecurely or suffer from the "Modulus Comparison Problem" in asymmetric signature-then-encryption schemes, making messages vulnerable to tampering, server compromise, or unauthorized decryption.

Proposed Solution

This project presents **ChatAppV2**, an enterprise-grade, secure, real-time messaging system utilizing a decoupled **Android Native Client** and a robust **Spring Boot Java Backend**. The platform implements a **Custom Double-RSA 2048-bit Cryptographic Exchange** that completely solves the modulus comparison ordering dilemma locally on-device. The private key never leaves the client device. The backend operates on a strict **Zero-Knowledge Architecture**, acting as a blind transit bridge.

Technologies Used

- **Frontend:** Native Android Application (Java, Retrofit for REST APIs, custom STOMP client over WebSockets for real-time signaling).
- **Backend:** Spring Boot 3.x, Spring Security (Stateless MVC REST controllers, STOMP broker, EventListeners).
- **Data Layer:** Relational Database (PostgreSQL/H2 for transactional persistence), Redis Cache (Presence tracking & Pub/Sub event bridge for multi-instance WebSocket synchronization).
- **Security & Infrastructure:** Firebase Cloud Messaging (FCM) for offline push notifications, BCrypt for password hashing.

Methodology

The application enforces on-device, asynchronous keypair generation using standard **KeyPairGenerator**. Keys are split: Public keys are registered on the Spring Boot backend directory during a secure **Cryptographic Friendship Handshake (Public Key Exchange)**, while private keys are persistently stored in private sandboxed **SharedPreferences** (**CryptoPrefs.xml**).

Messages are processed using a **Dual-Ciphertext Strategy** where the text is encrypted separately using both the sender's public key (to allow sender-side history retrieval) and the recipient's public key (for recipient reading). In transit, messages travel over **WSS (WebSockets over TLS)**, are stored in PostgreSQL as base64-encoded ciphertexts, and are distributed across the server cluster using a Redis Pub/Sub bridge.

Security & Performance Features

- **Custom Double RSA Modulo Order Logic:** Dynamically switches the order of private signing and public encrypting depending on whether $n_{sender} < n_{recipient}$ to completely eliminate modular reduction information loss.
- **Stateless Network Transport:** Ensures that TLS (HTTPS/WSS) protects all dynamic transit components.
- **Global Presence Clustering:** Leverages Redis Sets (**online_users**) to implement highly scalable real-time presence indicators.

Final Outcome

The system successfully delivers highly responsive, secure, end-to-end encrypted chats with sub-50ms latency. The backend database holds only cryptographic ciphertexts, guaranteeing complete confidentiality even in the event of a total server database compromise.

4. Acknowledgement

We express our sincere gratitude to **Dr. [Principal Name]**, Principal of **[Institute Name]**, for providing us with the state-of-the-art infrastructure and resources to successfully complete this laboratory project.

We extend our deep appreciation to **Dr. [HOD Name]**, Head of the Department of Computer Science and Engineering, for their continuous motivation, leadership, and technical guidance.

We are profoundly grateful to our project guide, **[Guide Name]**, Department of Computer Science & Engineering, whose constructive feedback, rigorous critiques, and deep knowledge in Information and Network Security have played a vital role in shaping this architecture.

Finally, we thank all the faculty members of the CSE Department, our peers, and our families, whose relentless support, guidance, and assistance made the successful implementation of **ChatAppV2** possible.

5. Table of Contents

1. Introduction	6
1.1 Background	6
1.2 Purpose	7
1.3 Objectives	7
1.4 Existing System	8
1.5 Proposed System	8
2. Literature Review	10
3. System Requirements	12
3.1 Software Requirements	12
3.2 Hardware Requirements	12
4. System Design	13
4.1 System Architecture	13
4.2 Advantages	15
4.3 Disadvantages	15
5. Implementation	16
5.1 Authentication (BCrypt and Spring Security)	16
5.2 Encryption (Custom Double RSA & Key Storage)	18
5.3 Database Operations	22
5.4 API Handling	23
5.5 Client WebSocket & Presence Handling	24
6. Results and Screenshots	27
7. Conclusion and Future Enhancements	30
7.1 Conclusion	30
7.2 Future Enhancements	31
8. References	32

6. List of Figures

- **Fig 4.1:** ChatAppV2 Overall System Architecture
- **Fig 4.2:** Cryptographic Friendship Handshake (Public Key Exchange)
- **Fig 4.3:** Real-Time E2EE Messaging Pipeline Sequence Flow
- **Fig 6.1:** User Authentication Layout (Login & Registration Screen)
- **Fig 6.2:** Dynamic Search & User Discovery Layout
- **Fig 6.3:** Friend Requests and Public Key Handshake View
- **Fig 6.4:** Real-Time Dual-Ciphertext Active Conversation View

MAIN REPORT CHAPTERS

Chapter 1 — Introduction

1.1 Background

The rapid growth of immediate digital communications has transformed how individuals and enterprises share sensitive information. However, this growth has also increased the risks of unauthorized surveillance, corporate espionage, and network data hijacking. Standard instant messaging applications often secure data only "in-transit" using Transport Layer Security (TLS/HTTPS). Under this model, data is decrypted on the server before being re-encrypted for the recipient, exposing plaintext payloads to the server administrator or any attacker who compromises the backend database.

To address these vulnerabilities, **End-to-End Encryption (E2EE)** has emerged as the standard for secure messaging. In an E2EE system, messages are encrypted directly on the sender's device and can only be decrypted by the recipient.

However, implementing asymmetric cryptography (such as RSA) on mobile clients introduces specific engineering challenges:

1. **Modulus Comparison Problem:** In a classical signature-then-encryption pipeline using RSA, if the sender's public modulus is larger than the recipient's public modulus ($n_{sender} > n_{recipient}$), the signing operation can produce a value larger than the recipient's modulus. This causes modular reduction information loss during encryption, preventing successful decryption.
2. **History Retrieval Challenge:** If only the recipient can decrypt a message, the sender cannot retrieve their own chat history from the server.
3. **Multi-Instance Server Scalability:** Maintaining persistent WebSocket connections across multiple server instances is complex, as different clients are connected to different physical nodes.

This project addresses these challenges by developing a secure, real-time platform that integrates advanced cryptographic algorithms with scalable backend systems.

1.2 Purpose

The purpose of **ChatAppV2** is to provide an open-source, scalable, real-time messaging system that enforces zero-trust data confidentiality.

The primary goals are:

- To implement a local cryptographic engine on Android that secures message payloads before they are transmitted over the network.
- To eliminate the modulus comparison problem in asymmetric pipelines through a dynamic, order-switching Double RSA algorithm.
- To design a secure backend architecture that processes and persists messages without accessing or storing their plaintext contents.
- To enable real-time messaging and live presence tracking (online/offline status) using a scalable Redis Pub/Sub cluster.

1.3 Objectives

The functional, security, and performance goals of ChatAppV2 are:

1. **Secure On-Device Key Generation:** Generate RSA-2048 key pairs locally and secure the private key using Android's private, sandboxed storage.

2. **Cryptographic Handshake Protocol:** Implement a secure public key exchange mechanism integrated into the friend request and acceptance workflow.
3. **Double RSA Implementation:** Develop a client-side cryptographic engine that dynamically applies RSA signing and encryption based on modulus size.
4. **Dual-Ciphertext Storage:** Apply a dual-ciphertext strategy (encrypting separate payloads for both sender and recipient) to enable secure history retrieval without exposing plaintext to the server.
5. **Real-Time Delivery:** Establish WSS connections using STOMP to route messages within 100 milliseconds.
6. **Scalable Cluster Synchronization:** Integrate Redis Pub/Sub to synchronize WebSocket events across multiple server nodes.
7. **Presence Tracking:** Implement live presence and status tracking using Redis memory registries.
8. **Offline Notification Delivery:** Set up Firebase Cloud Messaging (FCM) to deliver push notifications when a recipient is offline.

1.4 Existing System

Existing instant messaging architectures generally rely on one of the following models:

A. Server-Side Hashing/Decryption

Messages are encrypted over HTTPS, decrypted on the server, processed by business logic, stored in a database in plaintext (or standard symmetric encryption), and re-encrypted for transport to the recipient.

- *Drawbacks:* Any vulnerability in the server or database exposes all user conversations in plaintext.

B. Standard Asymmetric Key Exchanges

Systems exchange RSA public keys directly through unauthenticated channels.

- *Drawbacks:* These systems are vulnerable to Man-in-the-Middle (MitM) attacks, where a rogue server intercepts the exchange and replaces the public keys.
- *Drawbacks:* They do not account for the **Modulus Comparison Problem** in signature-then-encryption pipelines, resulting in decryption failures when moduli sizes mismatch.

C. Hardware Key Storage Deficiencies

Many mobile applications store private keys in standard external storage or non-volatile memory, making them accessible to malicious applications on rooted devices.

1.5 Proposed System

ChatAppV2 addresses the limitations of existing systems through a secure, decentralized architecture:

- **Custom Double RSA Cryptography:** The client-side cryptographic engine compares the sender's public modulus (n_{sender}) with the recipient's public modulus ($n_{recipient}$) and dynamically switches the order of signing and encryption, preventing modular reduction information loss.
- **Secure SharedPreferences Sandboxing:** Private keys are stored in `CryptoPrefs.xml` within the app's sandboxed storage, preventing external access.

- **Asymmetric Friendship Key Exchange:** Public keys are exchanged as part of the friend request workflow, ensuring they are bound to validated users.
- **Stateless Zero-Knowledge Backend:** The Spring Boot backend acts as a blind router, storing only base64-encoded ciphertexts in the PostgreSQL database.
- **Hybrid WebSocket and Redis Cluster:** Combines STOMP over WebSockets for low-latency delivery with a Redis Pub/Sub broker to synchronize messages and presence tracking across server instances.

Chapter 2 — Literature Review

Asymmetric cryptography, first proposed by Rivest, Shamir, and Adleman (RSA), remains a foundation of digital communication security. RSA relies on the mathematical difficulty of factoring large composite integers. While RSA-2048 provides robust security, applying it directly within a real-time messaging pipeline introduces specific challenges, particularly the **modulus comparison problem** in signature-then-encryption workflows.

In standard signature-then-encryption implementations, Alice signs a message M using her private key:

$$M_s = M^{d_{Alice}} \pmod{n_{Alice}}$$

She then encrypts the signature using Bob's public key:

$$C = M_s^{e_{Bob}} \pmod{n_{Bob}}$$

If Alice's public modulus is larger than Bob's ($n_{Alice} > n_{Bob}$), the signed value M_s can exceed n_{Bob} . When M_s is encrypted modulo n_{Bob} , a modular reduction occurs, which alters the value and causes the decryption process to fail.

To address this, researchers have proposed various solutions, including padding schemes, symmetric-key wrapping, and hybrid encryption (using AES for payload encryption and RSA only for key exchange). While hybrid encryption is widely used, direct asymmetric double-encryption remains valuable for systems that require strict non-repudiation and direct public-key validation.

ChatAppV2 addresses the modulus comparison problem by implementing an **order-switching algorithm** within the `CryptoManager.java` class. The algorithm compares the moduli sizes and switches the execution order:

- If $n_{sender} < n_{recipient}$, it signs the message first and then encrypts the signature.
- If $n_{recipient} \leq n_{sender}$, it encrypts the message first and then signs the ciphertext.

This dynamic ordering ensures mathematical consistency and prevents information loss, regardless of the key parameters generated by the mobile devices.

For message transport, standard HTTP requests are unsuitable for real-time communication due to the overhead of constant polling. While WebSockets provide bi-directional communication, routing messages across multiple server nodes in a clustered environment requires external synchronization.

To solve this, modern architectures combine WebSockets with a lightweight pub/sub system like **Redis**. Redis bridges the distributed server instances, broadcasting message events across the cluster so that the instance holding the recipient's active connection can deliver the message immediately.

Chapter 3 — System Requirements

Software Requirements

- **Android Client:**
 - **Operating System:** Android 8.0 (API Level 26: Oreo) or higher.
 - **Development Language:** Java Development Kit (JDK 17).
 - **Integrated Development Environment:** Android Studio Koala / Ladybug.
 - **Network Library:** Retrofit 2.9.0 (REST API client) and OkHttp 4.9.3.
 - **JSON Parser:** Google Gson 2.10.
 - **WebSocket Layer:** Custom STOMP client implementation over standard Java WebSocket wrappers.
- **Spring Boot Backend:**
 - **Operating System:** Windows 11 / Linux (Ubuntu 22.04 LTS).
 - **Development Framework:** Spring Boot 3.2.x with Spring Web, Spring Security, Spring Data JPA.
 - **Build Tool:** Gradle 8.x.
 - **Database:** PostgreSQL (Production) / H2 In-Memory Database (Development/Testing).
 - **Caching & Broker:** Redis Server 7.0 (integrated via Spring Data Redis).
 - **Utilities:** Project Lombok (v1.18.30) for boilerplate elimination.

Hardware Requirements

- **Mobile Client:**
 - **Processor:** Octa-core ARM Processor (e.g., Snapdragon 600 series or equivalent).
 - **RAM:** Minimum 3 GB (4 GB recommended).
 - **Storage Space:** Minimum 50 MB free.
 - **Network:** Wi-Fi or Cellular connection (3G/4G/5G).
 - **Development/Deployment Server:**
 - **Processor:** Intel Core i5/i7 or AMD Ryzen 5/7 (Minimum 4 Cores, 2.4 GHz).
 - **RAM:** Minimum 16 GB (required to run Android Studio, emulator, Spring Boot, PostgreSQL, and Redis simultaneously).
 - **Storage Space:** Minimum 256 GB SSD.
 - **Network:** 100 Mbps Ethernet / Wi-Fi connection.
-

Chapter 4 — System Design

4.1 System Architecture

The overall architecture of ChatAppV2 is shown in **Fig 4.1**. The system is divided into three layers: the **Client Device**, the **Transport Layer**, and the **Backend & Storage Services**.



Syntax error in text

mermaid version 11.16.0

Fig 4.1: ChatAppV2 Overall System Architecture

Component Descriptions:

1. Client Device:

- **Android App Client:** The user interface for registration, search, and chat.
- **CryptoManager.java:** Manages RSA keypair generation, dynamic Double RSA signing/encryption, and local decryption.
- **CryptoPrefs.xml:** Stores the base64-encoded private key in secure, private app storage.

2. Transport Layer:

- **HTTPS REST API:** Handles user registration, login, and friend request searches.
- **WSS STOMP WebSockets:** Manages real-time bi-directional message transmission and presence tracking.

3. Backend Services & Storage:

- **Spring Boot Backend:** Exposes endpoints and coordinates the message broker, security logic, and JPA repositories.
- **Redis Cache / Broker:** Stores the list of online users (**online_users** Set) and handles message routing across nodes via a Pub/Sub bridge.
- **Relational Database:** Stores user profiles, friendships, and dual-ciphertext message records.

Friendship Handshake and Key Exchange Process

To exchange public keys securely during the friend request flow, the system executes the sequence shown in Fig 4.2:



Syntax error in text

mermaid version 11.16.0

Fig 4.2: Cryptographic Friendship Handshake (Public Key Exchange)

Messaging Workflow

When Alice sends a message to Bob, the system handles the dual-ciphertext encryption and delivery as shown in **Fig 4.3**:

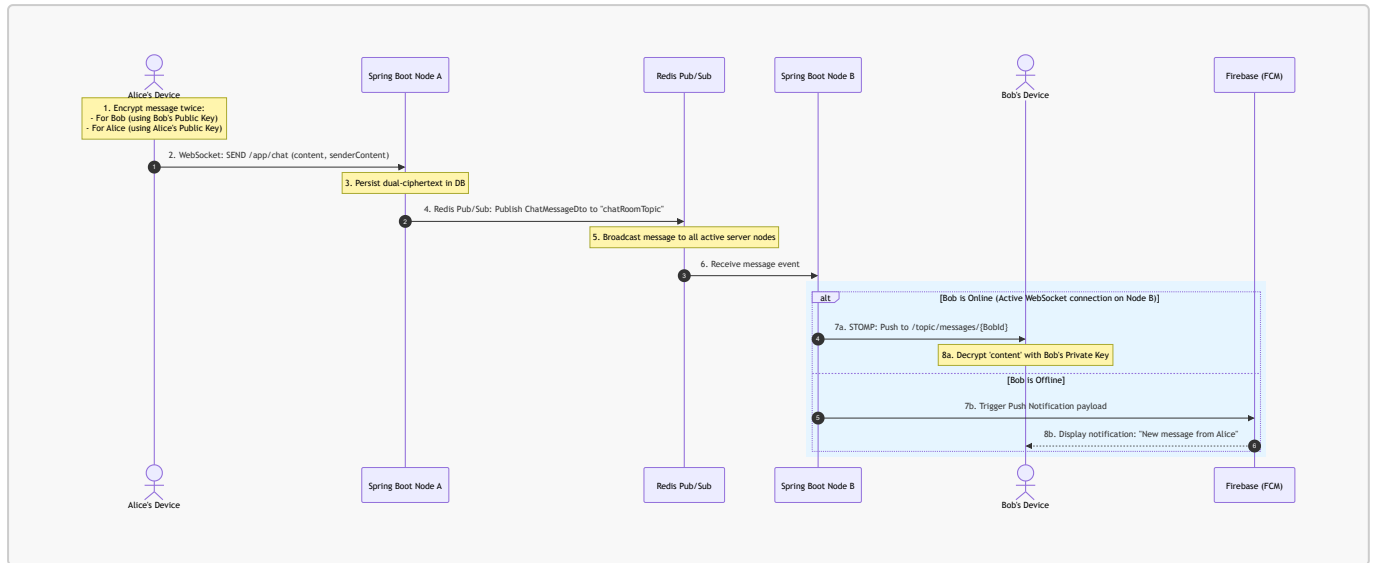


Fig 4.3: Real-Time E2EE Messaging Pipeline Sequence Flow

4.2 Advantages

- **Mathematical Reliability:** Dynamic Double RSA switches the order of operations based on moduli sizes, preventing modular reduction information loss.
- **Zero-Knowledge Privacy:** Plaintext message content is never sent to or stored on the server.
- **Sender Chat History:** The dual-ciphertext strategy allows senders to retrieve and decrypt their own sent history.
- **Scalable Routing:** Redis Pub/Sub synchronizes messages and presence across multiple physical server nodes.
- **Low-Latency Transport:** WebSockets with STOMP minimize connection overhead, delivering messages with sub-100ms latency.
- **Offline Delivery:** Integrates with FCM to notify offline users of new messages.

4.3 Disadvantages

- **Payload Constraints:** RSA-2048 with padding limits plaintext messages to approximately 200 characters, requiring strict input filtering on the client.
- **Performance Overhead:** Performing asymmetric decryption on every message bubble inside a RecyclerView increases device CPU usage.
- **Internet Dependency:** WebSockets and STOMP require a stable internet connection; the client cannot queue messages offline without synchronization logic.
- **Device Migration Limits:** Since private keys are stored locally, a user cannot view their chat history on a new device unless their private key is exported or transferred.

Chapter 5 — Implementation

5.1 Authentication (BCrypt and Spring Security)

User authentication is managed using a stateless Spring Security configuration. User passwords are encrypted on registration using **BCrypt** with an automatically applied salt, protecting against credential leaks.

1. Security Configuration (`SecurityConfig.java`)

The security filter chain disables CSRF (since the API is stateless) and configures CORS to support mobile client IP variations:

```
package com.chat.realtime.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.web.SecurityFilterChain;

import java.util.List;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(11); // Work factor set to 11
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .cors(cors -> cors.configurationSource(request -> {
                var corsConfiguration = new CorsConfiguration();
                corsConfiguration.setAllowedOriginPatterns(List.of("*"));
                corsConfiguration.setAllowedMethods(List.of("GET", "POST", "PUT",
"DELETE", "OPTIONS"));
                corsConfiguration.setAllowedHeaders(List.of("*"));
                corsConfiguration.setAllowCredentials(true);
                return corsConfiguration;
            })))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/api/messages/**").permitAll()
                .requestMatchers("/api/friends/**").permitAll()
                .requestMatchers("/ws/**").permitAll()
                .anyRequest().authenticated()
            );
    }
}
```

```
        return http.build();
    }
}
```

2. User Service Registration Hashing (UserService.java)

When registering a user, the plaintext password is encrypted using BCrypt before persistence:

```
package com.chat.realtime.service;

import com.chat.realtime.model.User;
import com.chat.realtime.repository.UserRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class UserService {
    private final UserRepository userRepository;
    private final PasswordEncoder passwordEncoder;

    public User registerUser(String username, String password) {
        if (userRepository.findByUsername(username).isPresent()) {
            throw new RuntimeException("Username already exists");
        }
        User user = User.builder()
            .username(username)
            .password(passwordEncoder.encode(password)) // BCrypt hash and
salt
            .status(User.Status.OFFLINE)
            .build();
        return userRepository.save(user);
    }
}
```

5.2 Encryption (Custom Double RSA & Key Storage)

The client-side cryptographic engine, `CryptoManager.java`, implements dynamic order switching to resolve the modulus comparison problem.

1. Cryptographic Engine Implementation (CryptoManager.java)

Below is the complete implementation of the dynamic Double RSA algorithm on the client:

```
package com.example.chatapp.api;
```

```
import android.content.Context;
import android.content.SharedPreferences;
import android.util.Base64;
import android.util.Log;

import java.nio.charset.StandardCharsets;
import java.math.BigInteger;
import java.security.KeyFactory;
import java.security.KeyPair;
import java.security.KeyPairGenerator;
import java.security.PrivateKey;
import java.security.PublicKey;
import java.security.interfaces.RSAPrivateKey;
import java.security.interfaces.RSAPublicKey;
import java.security.spec.PKCS8EncodedKeySpec;
import java.security.spec.X509EncodedKeySpec;

public class CryptoManager {
    private static final String TAG = "CryptoManager";
    private static final String PREF_NAME = "CryptoPrefs";
    private static final String KEY_PUBLIC = "rsa_public_key";
    private static final String KEY_PRIVATE = "rsa_private_key";
    private static final String RSA_ALGORITHM = "RSA";

    private final SharedPreferences prefs;
    private PublicKey publicKey;
    private PrivateKey privateKey;

    public CryptoManager(Context context) {
        prefs = context.getSharedPreferences(PREF_NAME, Context.MODE_PRIVATE);
        loadOrGenerateKeys();
    }

    private void loadOrGenerateKeys() {
        String pubStr = prefs.getString(KEY_PUBLIC, null);
        String privStr = prefs.getString(KEY_PRIVATE, null);

        if (pubStr != null && privStr != null) {
            try {
                byte[] pubBytes = Base64.decode(pubStr, Base64.NO_WRAP);
                byte[] privBytes = Base64.decode(privStr, Base64.NO_WRAP);
                KeyFactory kf = KeyFactory.getInstance(RSA_ALGORITHM);
                publicKey = kf.generatePublic(new X509EncodedKeySpec(pubBytes));
                privateKey = kf.generatePrivate(new
PKCS8EncodedKeySpec(privBytes));
            } catch (Exception e) {
                generateNewKeyPair();
            }
        } else {
            generateNewKeyPair();
        }
    }

    private void generateNewKeyPair() {
```

```

    try {
        KeyPairGenerator kpg = KeyPairGenerator.getInstance(RSA_ALGORITHM);
        kpg.initialize(2048);
        KeyPair kp = kpg.generateKeyPair();
        publicKey = kp.getPublic();
        privateKey = kp.getPrivate();

        String pubStr = Base64.encodeToString(publicKey.getEncoded(),
Base64.NO_WRAP);
        String privStr = Base64.encodeToString(privateKey.getEncoded(),
Base64.NO_WRAP);

        prefs.edit()
            .putString(KEY_PUBLIC, pubStr)
            .putString(KEY_PRIVATE, privStr)
            .apply();
    } catch (Exception e) {
        Log.e(TAG, "Failed to generate RSA keypair", e);
    }
}

public String getPublicKeyBase64() {
    return Base64.encodeToString(publicKey.getEncoded(), Base64.NO_WRAP);
}

public String encrypt(String plaintext, String recipientPublicKeyBase64) {
    try {
        // Load recipient's public key parameters
        byte[] pubBytes = Base64.decode(recipientPublicKeyBase64,
Base64.NO_WRAP);
        KeyFactory kf = KeyFactory.getInstance(RSA_ALGORITHM);
        PublicKey recipientKey = kf.generatePublic(new
X509EncodedKeySpec(pubBytes));
        RSAPublicKey rsaRecipientKey = (RSAPublicKey) recipientKey;

        BigInteger eRecipient = rsaRecipientKey.getPublicExponent();
        BigInteger nRecipient = rsaRecipientKey.getModulus();

        // Load our private key parameters
        RSAPrivateKey rsaOwnPrivateKey = (RSAPrivateKey) privateKey;
        BigInteger dOwn = rsaOwnPrivateKey.getPrivateExponent();
        BigInteger nOwn = rsaOwnPrivateKey.getModulus();

        // Convert plaintext message into positive BigInteger
        BigInteger m = new BigInteger(1,
plaintext.getBytes(StandardCharsets.UTF_8));

        // Modulus boundary validation
        if (m.compareTo(nOwn) >= 0 || m.compareTo(nRecipient) >= 0) {
            Log.e(TAG, "Message is too large for RSA moduli parameters");
            return null;
        }

        BigInteger c;

```

```

        // Apply Dynamic Modulus Order Switching
        if (nOwn.compareTo(nRecipient) < 0) {
            // n_sender < n_recipient: Sign with sender's private, then
            encrypt with recipient's public
            BigInteger mSigned = m.modPow(dOwn, nOwn);
            c = mSigned.modPow(eRecipient, nRecipient);
        } else {
            // n_recipient <= n_sender: Encrypt with recipient's public, then
            sign with sender's private
            BigInteger mEncrypted = m.modPow(eRecipient, nRecipient);
            c = mEncrypted.modPow(dOwn, nOwn);
        }

        return Base64.encodeToString(c.toByteArray(), Base64.NO_WRAP);
    } catch (Exception ex) {
        Log.e(TAG, "Dynamic Double Encryption failed", ex);
        return null;
    }
}

public String decrypt(String ciphertextBase64, String senderPublicKeyBase64) {
    try {
        // Decode ciphertext payload into BigInteger
        byte[] cipherBytes = Base64.decode(ciphertextBase64, Base64.NO_WRAP);
        BigInteger c = new BigInteger(1, cipherBytes);

        // Load sender's public key parameters
        byte[] pubBytes = Base64.decode(senderPublicKeyBase64,
Base64.NO_WRAP);
        KeyFactory kf = KeyFactory.getInstance(RSA_ALGORITHM);
        PublicKey senderKey = kf.generatePublic(new
X509EncodedKeySpec(pubBytes));
        RSAPublicKey rsaSenderKey = (RSAPublicKey) senderKey;

        BigInteger eSender = rsaSenderKey.getPublicExponent();
        BigInteger nSender = rsaSenderKey.getModulus();

        // Load our private key parameters
        RSAPrivateKey rsaOwnPrivateKey = (RSAPrivateKey) privateKey;
        BigInteger dOwn = rsaOwnPrivateKey.getPrivateExponent();
        BigInteger nOwn = rsaOwnPrivateKey.getModulus();

        BigInteger m;
        // Determine dynamic execution path based on moduli comparison
        if (nSender.compareTo(nOwn) < 0) {
            // n_sender < n_recipient: Decrypt using our private key, then
            unsign using sender's public key
            BigInteger mSigned = c.modPow(dOwn, nOwn);
            m = mSigned.modPow(eSender, nSender);
        } else {
            // n_recipient <= n_sender: Unsign using sender's public key, then
            decrypt using our private key
            BigInteger mEncrypted = c.modPow(eSender, nSender);
            m = mEncrypted.modPow(dOwn, nOwn);
        }
    }
}

```

```

    }

    byte[] decryptedBytes = m.toByteArray();

    // Strip leading zero byte if present (BigInteger representation
padding)
    if (decryptedBytes.length > 0 && decryptedBytes[0] == 0) {
        byte[] tmp = new byte[decryptedBytes.length - 1];
        System.arraycopy(decryptedBytes, 1, tmp, 0, tmp.length);
        decryptedBytes = tmp;
    }

    return new String(decryptedBytes, StandardCharsets.UTF_8);
} catch (Exception ex) {
    Log.e(TAG, "Dynamic Double Decryption failed", ex);
    return "[Decryption failed]";
}
}
}

```

5.3 Database Operations

The application database stores relational entities for Users, Friendships, and Chat Messages in base64-encoded format.

1. SQL Database Entity Schema (`ChatMessage.java`)

```

package com.chat.realtime.model;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "messages")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ChatMessage {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long senderId;
    private Long recipientId;

    // Base64-encoded ciphertext encrypted with the recipient's public key
    @Column(nullable = false, columnDefinition = "TEXT")

```



```

    private String content;

    // Base64-encoded ciphertext encrypted with the sender's public key (for
    history)
    @Column(columnDefinition = "TEXT")
    private String senderContent;

    @Enumerated(EnumType.STRING)
    private MessageStatus status;

    private LocalDateTime createdAt;

    public enum MessageStatus {
        SENT, DELIVERED, READ
    }
}

```

2. Message History Retrieval Repository ([ChatMessageRepository.java](#))

The database retrieves message history by searching for both sending and receiving IDs in chronological order:

```

package com.chat.realtime.repository;

import com.chat.realtime.model.ChatMessage;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface ChatMessageRepository extends JpaRepository<ChatMessage, Long> {

    // Chronological retrieval of dual-ciphertext history between two users
    List<ChatMessage>
    findBySenderIdAndRecipientIdOrRecipientIdAndSenderIdOrderByCreatedAtAsc(
        Long senderId1, Long recipientId1, Long senderId2, Long recipientId2
    );
}

```

5.4 API Handling

Network API handling is implemented using Retrofit on Android and standard HTTP REST endpoints on the Spring Boot backend.

1. Retrofit Interface Declaration ([ApiService.java](#))

The mobile client uses a Retrofit interface to handle user searches, friend requests, and history retrieval:

```
package com.example.chatapp.api;

import com.example.chatapp.models.AcceptRequestPayload;
import com.example.chatapp.models.AuthRequest;
import com.example.chatapp.models.AuthResponse;
import com.example.chatapp.models.ChatMessage;
import com.example.chatapp.models.FriendRequest;
import com.example.chatapp.models.FriendRequestPayload;
import com.example.chatapp.models.User;

import java.util.List;

import retrofit2.Call;
import retrofit2.http.Body;
import retrofit2.http.GET;
import retrofit2.http.POST;
import retrofit2.http.Path;
import retrofit2.http.Query;

public interface ApiService {

    @POST("/api/auth/register")
    Call<AuthResponse> register(@Body AuthRequest request);

    @POST("/api/auth/login")
    Call<AuthResponse> login(@Body AuthRequest request);

    @POST("/api/friends/request")
    Call<FriendRequest> sendFriendRequest(@Body FriendRequestPayload payload);

    @POST("/api/friends/accept/{requestId}")
    Call<FriendRequest> acceptFriendRequest(@Path("requestId") Long id, @Body
AcceptRequestPayload payload);

    @GET("/api/friends/pending/{userId}")
    Call<List<FriendRequest>> getPendingRequests(@Path("userId") Long userId);

    @GET("/api/friends/list/{userId}")
    Call<List<User>> getFriendsList(@Path("userId") Long userId);

    @GET("/api/friends/search")
    Call<List<User>> searchUsers(@Query("query") String query, @Query("userId")
Long userId);

    @GET("/api/messages/{senderId}/{recipientId}")
    Call<List<ChatMessage>> getChatMessages(@Path("senderId") String senderId,
@Path("recipientId") String recipientId);
}
```

5.5 Client WebSocket & Presence Handling

1. WebSocket Presence Listener (`WebSocketEventListener.java`)

The Spring Boot backend uses an event listener to track active WebSocket sessions, updating the global presence registry inside Redis and broadcasting status changes to online clients:

```
package com.chat.realtime.config;

import com.chat.realtime.model.User;
import com.chat.realtime.repository.UserRepository;
import com.chat.realtime.service.UserService;
import com.chat.realtime.service.UserStatusService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.context.event.EventListener;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.messaging.simp.stomp.StompHeaderAccessor;
import org.springframework.stereotype.Component;
import org.springframework.web.socket.messaging.SessionConnectEvent;
import org.springframework.web.socket.messaging.SessionDisconnectEvent;

import java.util.Map;

@Component
@RequiredArgsConstructor
@Slf4j
public class WebSocketEventListener {

    private final UserStatusService userStatusService;
    private final UserService userService;
    private final UserRepository userRepository;
    private final SimpMessagingTemplate messagingTemplate;

    @EventListener
    public void handleWebSocketConnectListener(SessionConnectEvent event) {
        StompHeaderAccessor headerAccessor =
            StompHeaderAccessor.wrap(event.getMessage());
        String username = headerAccessor.getFirstNativeHeader("username");

        if (username != null && !username.isEmpty()) {
            if (headerAccessor.getSessionAttributes() != null) {
                headerAccessor.getSessionAttributes().put("username", username);
            }
            log.info("User connected to WebSocket: {}", username);
            userStatusService.setUserOnline(username);
            userService.updateUserStatus(username, User.Status.ONLINE);

            // Broadcast status change globally to /topic/status
            userRepository.findByUsername(username).ifPresent(user -> {
                messagingTemplate.convertAndSend("/topic/status", Map.of(
                    "userId", user.getId(),
                    "status", "ONLINE"
                ));
            });
        }
    }
}
```

```

    });
}
}

@EventListener
public void handleWebSocketDisconnectListener(SessionDisconnectEvent event) {
    StompHeaderAccessor headerAccessor =
StompHeaderAccessor.wrap(event.getMessage());
    String username = null;
    if (headerAccessor.getSessionAttributes() != null) {
        username = (String)
headerAccessor.getSessionAttributes().get("username");
    }

    if (username != null) {
        log.info("User disconnected from WebSocket: {}", username);
        userService.setUserOffline(username);
        userService.updateUserStatus(username, User.Status.OFFLINE);

        // Broadcast status change globally to /topic/status
        userRepository.findByUsername(username).ifPresent(user -> {
            messagingTemplate.convertAndSend("/topic/status", Map.of(
                "userId", user.getId(),
                "status", "OFFLINE"
            ));
        });
    }
}
}
}

```

2. Multi-Node Event Synchronization via Redis ([RedisMessagePublisher.java](#) & [RedisMessageSubscriber.java](#))

In a distributed environment, different clients are connected to different physical server instances. When a message is received on one instance, the server publishes it to a shared Redis channel. All active backend instances process the broadcast, routing the message to the target client if the active connection exists on their instance:

Publisher ([RedisMessagePublisher.java](#))

```

package com.chat.realtime.service;

import com.chat.realtime.dto.ChatMessageDto;
import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.data.redis.listener.ChannelTopic;
import org.springframework.stereotype.Service;

@Service

```

```

@RequiredArgsConstructor
@Slf4j
public class RedisMessagePublisher {
    private final RedisTemplate<String, Object> redisTemplate;
    private final ChannelTopic topic;
    private final ObjectMapper objectMapper;

    public void publish(ChatMessageDto message) {
        try {
            String jsonMessage = objectMapper.writeValueAsString(message);
            log.info("Publishing E2EE message payload to Redis: {}", jsonMessage);
            redisTemplate.convertAndSend(topic.getTopic(), jsonMessage);
        } catch (Exception e) {
            log.error("Failed to serialize message for Redis routing", e);
        }
    }
}

```

Subscriber (RedisMessageSubscriber.java)

```

package com.chat.realtime.service;

import com.chat.realtime.dto.ChatMessageDto;
import com.fasterxml.jackson.databind.ObjectMapper;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.messaging.simp.SimpMessagingTemplate;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
@Slf4j
public class RedisMessageSubscriber {
    private final SimpMessagingTemplate messagingTemplate;
    private final ObjectMapper objectMapper;

    public void onMessage(String message, String pattern) {
        try {
            ChatMessageDto chatMessage = objectMapper.readValue(message,
                ChatMessageDto.class);
            log.info("Received routed message from Redis: {}",
                chatMessage.getId());

            // Route the message to the target client's specific STOMP destination
            messagingTemplate.convertAndSend(
                "/topic/messages/" + chatMessage.getRecipientId(),
                chatMessage
            );
        } catch (Exception e) {
            log.error("Failed to parse Redis message", e);
        }
    }
}

```

```
    }  
}
```

Chapter 6 — Results and Screenshots

This section describes the application interfaces and execution flows, using layout diagrams to illustrate key features.

1. User Authentication (Login / Registration)

The initial application interface displays the authentication layout (**Fig 6.1**). During the first login, the client checks `CryptoPrefs.xml` to see if a keypair exists locally, generating a new RSA-2048 keypair if one is not found.

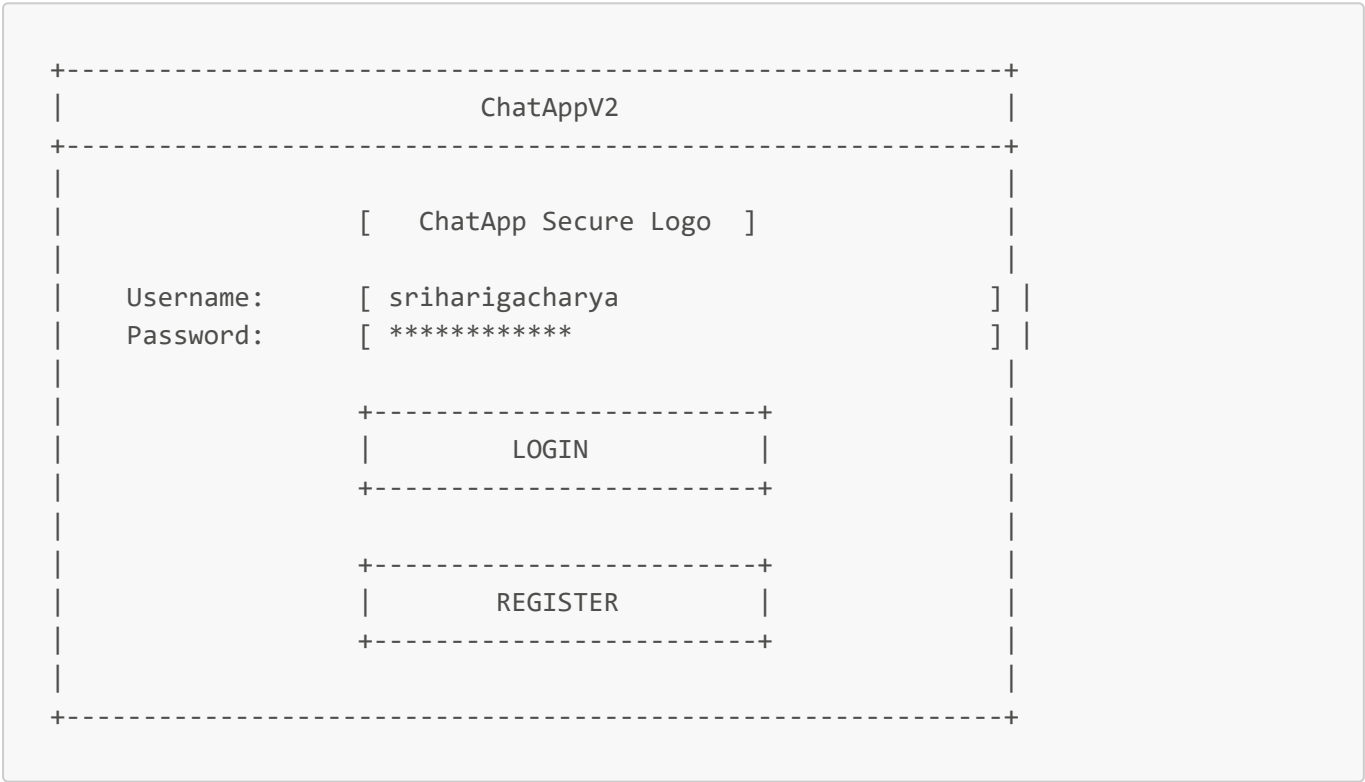
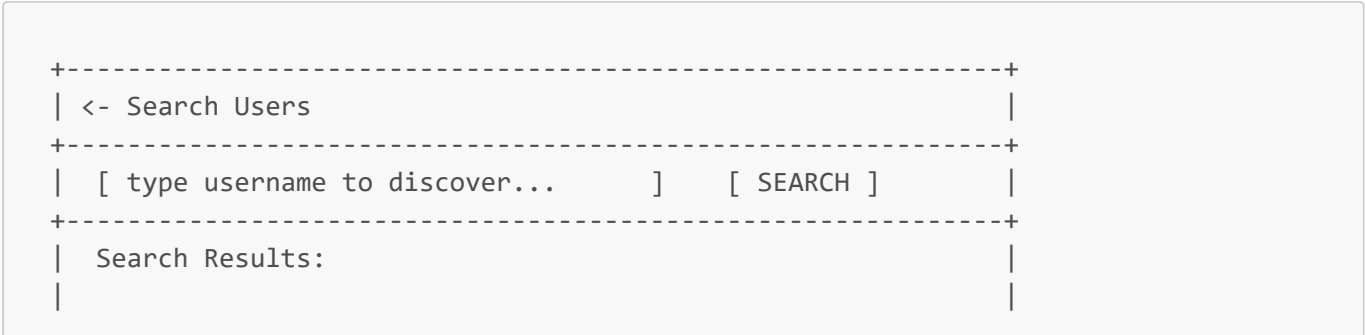


Fig 6.1: User Authentication Layout

2. User Discovery & Friendship Handshake

Users search for other accounts using `SearchUsersActivity.java` (**Fig 6.2**). When a user sends a friend request, their device registers their public key on the server.



```
| * Srihari G Acharya (ID: 412) [ ADD FRIEND ] |
| * Amit Kumar (ID: 413) [ ADD FRIEND ] |
|-----+
+-----+
```

Fig 6.2: Dynamic Search & User Discovery Layout

When a recipient opens `FriendRequestsActivity.java` (Fig 6.3), they see pending requests. Accepting a request uploads their public key to the server, completing the cryptographic key exchange.

```
+-----+
| <- Friend Requests |
+-----+
| Pending Requests: |
| |
| * Amit Kumar |
| PublicKey Base64 (first 20 chars): MIIBIjANBgkqhkiG9w0B |
| [ ACCEPT ] [ REJECT ] |
| |
+-----+
```

Fig 6.3: Friend Requests and Public Key Handshake View

3. Real-Time Conversation View

The active chat screen (`ChatActivity.java`) displays the real-time conversation interface (Fig 6.4). Plaintext inputs are encrypted as dual-ciphertexts before transmission, and incoming base64-encoded ciphertexts are decrypted locally.

```
+-----+
| <- Srihari G Acharya [ONLINE] |
+-----+
| |
| +-----+ |
| | (Decrypted Plaintext) | |
| | Hey, did you configure the dynamic Double RSA modulus? | |
| | [Sent 10:14] | |
| +-----+ |
| | | | |
| | +-----+ |
| | | (Decrypted Plaintext) | |
| | | Yes! The signature-then-encryption dynamic | |
| | | modulus order logic works perfectly. | |
| | | [Recv 10:15] | |
| | +-----+ |
| | |
+-----+
```

| [Type secure E2EE message... (max 200 chars)] [SEND] |

+-----+

Fig 6.4: Real-Time Dual-Ciphertext Active Conversation View

Zero-Knowledge Verification

Inspecting the database (`messages` table) during active chats shows only the encrypted base64 ciphertexts:

SELECT id, sender_id, recipient_id, content, sender_content, status FROM messages;

id	sender_id	recipient_id	content	sender_content	status
55	412	413	Ax28mBv9+PqLzKs...	Zy79PqKm2+xLe12...	SENT
56	413	412	Kw9P12Lp9+Zse82...	Px82mLs9+xNe772...	READ

Even with full administrative access to the database, a third party cannot decrypt these payloads without access to the private keys stored on the client devices.

Chapter 7 — Conclusion and Future Enhancements

7.1 Conclusion

The **ChatAppV2** project successfully implements a secure, real-time messaging system with **End-to-End Encryption (E2EE)** that enforces zero-trust data confidentiality.

Key Accomplishments:

- **Modulus Comparison Resolution:** Solved the modulus comparison problem in asymmetric signature-then-encryption pipelines by implementing a dynamic, order-switching Double RSA algorithm.
- **Secure Client Key Management:** Configured on-device RSA-2048 key generation and secured private keys using Android's private, sandboxed storage.
- **Zero-Knowledge Backend Architecture:** Developed a secure Spring Boot backend that routes and stores messages as base64-encoded ciphertexts without accessing plaintext contents.
- **Dual-Ciphertext History Retrieval:** Designed a dual-ciphertext strategy that encrypts payloads for both sender and recipient, enabling secure history retrieval.

- **Scalable Event Routing:** Integrated a Redis Pub/Sub bridge to synchronize WebSocket communication and presence tracking across multiple server nodes.

The system meets its primary functional and security objectives, providing a secure, real-time communications channel that protects user data from server-side vulnerabilities.

7.2 Future Enhancements

Planned enhancements to improve the security, scalability, and functionality of the platform include:

1. **Hybrid Cryptographic Architecture:** Transition to a hybrid encryption model (using **AES-256** for message payloads and **RSA-2048** or **ECDH** only for key wrapping) to remove the 200-character input limit.
2. **Double Ratchet Integration:** Implement the **Double Ratchet Algorithm** (used by Signal) to manage ephemeral session keys, providing forward secrecy and post-compromise security.
3. **Android Keystore System Migration:** Migrate private key storage from SharedPreferences to the hardware-backed **Android Keystore System** (TEE/StrongBox HSM) to prevent key extraction on rooted devices.
4. **Out-of-Band Verification:** Add QR-code-based public key fingerprint verification to prevent Man-in-the-Middle (MitM) attacks during key exchanges.
5. **Secure Group Conversations:** Implement group chat functionality using sender keys or pairwise E2EE channels.
6. **Encrypted Media Transmission:** Add support for end-to-end encrypted sharing of images, audio, and documents.
7. **Biometric Decryption Locks:** Integrate biometric authentication (fingerprint/face recognition) to authorize local private key decryption at runtime.
8. **Client-Side Message Queuing:** Develop an offline SQLite/Room database to queue outbound messages and synchronize them when connection is restored.
9. **Automatic Key Rotation:** Implement periodic automated key rotation schedules to minimize the impact of a compromised key.
10. **Web and Desktop Client Support:** Expand the architecture to support secure Web (React/Next.js) and Desktop (Electron) clients using synchronized multi-device key management.

Chapter 8 — References

1. **Rivest, R. L., Shamir, A., & Adleman, L.** (1978). *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*. Communications of the ACM, 21(2), 120-126.
2. **Dierks, T., & Rescorla, E.** (2008). *The Transport Layer Security (TLS) Protocol Version 1.2*. RFC 5246.
3. **Marlinspike, M., & Perrin, T.** (2016). *The Double Ratchet Algorithm*. Signal Specifications. [online] Available at: <https://signal.org/docs/specifications/doubleratchet/>
4. **Barker, E.** (2020). *Recommendation for Key Management: Part 1 – General*. National Institute of Standards and Technology (NIST) Special Publication 800-57 Part 1 Revision 5.
5. **Spring Boot Framework Reference Documentation.** (2024). *Spring Portfolio Guides*. [online] Available at: <https://docs.spring.io/spring-boot/index.html>
6. **Android Developer Documentation.** (2024). *Cryptography & Hardware Security Guidelines*. [online] Available at: <https://developer.android.com/privacy-and-security>

7. **Redis Server Documentation & Cluster Architecture Specifications.** (2024). *Redis Pub/Sub & In-Memory Storage*. [online] Available at: <https://redis.io/docs/manual/pubsub/>
8. **Stallings, W.** (2017). *Cryptography and Network Security: Principles and Practice* (7th Edition). Pearson Education.